Clase 12 · connect(), puertos efímeros y las colas de listen()

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 6 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Reintentos en connect()

Según la especificación de The Open Group, `connect()` devuelve **0** si se conecta y **−1** si falla (causa en `errno`). Cliente robusto que espera a que el servidor arranque:

```
while (connect(fd, (struct sockaddr *)&servidor, sizeof(servidor)) == -1) {
    printf("Servidor no disponible, reintentando en 5 segundos...\n");
    sleep(5);
}
```

Error común: omitir `sleep()`: ciclo infinito que satura red y CPU con miles de intentos por segundo.

Pruebas en vivo: un alumno corrió el cliente desde casa contra 132.248.59.6 y el servidor imprimió su IP pública. Otro abrió la IP y el puerto en el navegador: llegaron cabeceras HTTP (`GET / HTTP/1.1`, `User-Agent`) y el navegador no cerró la sesión limpia.

2. Puertos efímeros

> **Definición**
>
> Puertos temporales que el **SO del cliente** asigna dinámicamente a cada conexión saliente, para distinguir conversaciones que salen de la misma IP.
>
> Con `ss` se vieron tres conexiones desde la IP de PC Puma (132.247.100.226) con puertos de salida **49802, 48469 y 49808**. Al cerrar, el puerto se libera. Error común: confundir el puerto fijo de escucha del servidor (4898) con el efímero del cliente.

3. listen(), Three-Way Handshake y las dos colas

Paso	Cliente	Servidor (kernel)
1. **SYN**	Envía SYN → SYN_SENT	Recibe SYN → SYN_RECV; guarda la petición semiabierta en la **cola SYN**
2. **SYN-ACK**	Lo recibe → ESTABLISHED	Responde SYN-ACK
3. **ACK**	Envía ACK	Marca ESTABLISHED; pasa la conexión de la cola SYN a la **cola ACCEPT**

`accept()` extrae la primera conexión de la **cola ACCEPT** y devuelve el socket del cliente. Error común: creer que `listen()` maneja una sola cola.

4. La regla n + 1

La función del kernel que revisa si la cola de aceptación está llena (`sk_acceptq_is_full()`) usa **"mayor que" estricto**:

```
if (longitud_actual > backlog) { /* cola llena: rechazar */ }
```

> **Regla de examen**
>
> Con `listen(fds, 1)`: la primera encolada evalúa 1 > 1 (falso) y entra; solo se considera llena cuando 2 > 1. Se encolan **n + 1** conexiones pendientes (backlog 1 → 2).

5. Tarea 4: sysctl, SYN flood y monitoreo

```
sudo sysctl -w net.ipv4.tcp_syncookies=0   # desactiva las SYN cookies
sudo sysctl -w net.ipv4.tcp_syn_retries=1  # 1 reintento de SYN (2 intentos)
watch -n1 "ss -atnp"                       # Recv-Q encoladas, Send-Q backlog máximo
nc -v <IP> <puerto>                        # clientes en paralelo con netcat
```

- `tcp_syn_retries = 1` reduce el bloqueo de 63–180 s a unos instantes.

> **SYN flood**
>
> Ataque DoS: miles de SYN con IP falsificada (spoofing) que nunca mandan el ACK final; el servidor reserva memoria en la cola SYN para cada uno hasta agotarla y rechaza a usuarios legítimos. Las **SYN cookies** evitan reservar memoria para peticiones semiabiertas.
>
> En `ss`, para un socket en LISTEN: **Recv-Q** = conexiones encoladas ahora; **Send-Q** = longitud máxima del backlog. Cambiar sysctl requiere `sudo`.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

connect(). 0 éxito, −1 error; reintentar en `while` con `sleep(5)`.

Puertos efímeros. Los asigna el SO del cliente; misma IP, puertos distintos.

> **Handshake y colas (examen)**
> SYN → semiconexión en la **cola SYN**; SYN-ACK; ACK → pasa a la **cola ACCEPT**, de donde la saca `accept()`. Regla **n + 1** (el kernel compara con `>`).

Concepto	Para recordar
SYN flood	SYN falsos sin ACK llenan la cola SYN
SYN cookies	No reservan memoria para semiabiertas
`tcp_syncookies=0`, `tcp_syn_retries=1`	Para la Tarea 4
`ss -atnp`	Recv-Q encoladas / Send-Q backlog
`nc -v`	Simular clientes